Is `std::execution` a Universal Async Model?

Document Number:	P4041R0
Date:	2026-03-01
Audience:	LEWG

Reply-to:	Vinnie Falco vinnie.falco@gmail.com

Table of Contents

Abstract

Revision History

- R0: March 2026 (post-Croydon mailing)

1. Disclosure

2. Two Standard Async Models

3. What `std::execution` Does Well

4. The Post-Adoption Record

- 4.1 Changes by Category
- 4.2 Direction of Change

5. Published Findings

6. Structural Observations

- 6.1 The Error Channel Timeline

7. Established Practice

8. Ecosystem Adoption

9. Open Question

Acknowledgements

- References
  - WG21 Papers
  - LWG Issues
  - NB Ballot Comments
  - Blog Posts
  - Libraries

Platform Documentation

Other

Abstract

C++20 standardized coroutines. C++26 adds `std::execution`. Both are asynchronous models. The question is whether the C++26 model subsumes the C++20 model, or whether they serve different domains. This paper collects publicly available evidence.

Revision History

R0: March 2026 (post-Croydon mailing)

- Initial version.

---

1. Disclosure

The author developed P4003R0[36] ("Coroutines for I/O"), P4007R0[33] ("Senders and Coroutines"), P4014R0[34] ("The Sender Sub-Language"), and P2583R1[35] ("Symmetric Transfer and Sender Composition"). The author publishes Capy and Corosio, coroutine-native I/O libraries.

A coroutine-only design cannot express compile-time work graphs, does not support heterogeneous dispatch, and assumes cooperative scheduling. This paper does not cite those libraries or P4003R0 as evidence for any claim. They are disclosed as context for the author's perspective.

---

2. Two Standard Async Models

C++20 standardized coroutines (P0912R5[3], P0913R1[4]). They are ISO C++.

C++26 adds `std::execution` (P2300R10[1], "std::execution"). Schedulers, senders, receivers, and composable algorithms for asynchronous computation.

Both are standard facilities. Both are async models. The question is the relationship between them.

Eric Niebler wrote in "Structured Concurrency"[46] (2020):

> *"I think that 90% of all async code in the future should be coroutines simply for maintainability."*

Niebler wrote in "What Are Senders Good For, Anyway?"[47] (2024):

> *"If your library exposes asynchrony, then returning a sender is a great choice: your users can await the sender in a coroutine if they like."*

The framework's architect placed 90% of async code in the coroutine column.

---

3. What `std::execution` Does Well

`std::execution` provides zero-allocation sender pipelines for compile-time work graphs. The `connect` / `start` protocol collapses a sender chain into a single operation state with no heap allocation and no virtual dispatch. Domain customization allows a CUDA scheduler to customize `bulk` to emit a kernel launch instead of a loop. Type-level completion signatures enable compile-time routing without runtime inspection. NVIDIA's CUDA compiler does not support C++20 coroutines in device code[58]; stdexec[49] - the reference implementation - was built at NVIDIA.

Herb Sutter reported[48] that Citadel Securities uses `std::execution` in production: ***"We already use C++26's `std::execution` in production for an entire asset class, and as the foundation of our new messaging infrastructure."***

---

4. The Post-Adoption Record

P2300R10[1] was adopted into the C++26 working draft at St. Louis in July 2024. The following is our attempt at a complete enumeration of post-adoption changes, compiled from published WG21 mailings[61], the LWG issues list[60], and the C++26 NB ballot repository[62].

4.1 Changes by Category

Category	Papers / Issues	Count
Removals	P3682R0[12], P3149R11[21] (replacing `ensure_started`)	2
Rewrites	P3481R5[13] (`bulk`)	1
Architectural fixes	P3887R1[14], P3557R3[16]	2
Wording omnibus	P3396R1[15]	1
Post-adoption adds	P3433R1[17], P3284R4[18], P3570R2[19], P3552R3[6], P2079R10[20], P3149R11[21], P3927R0[22]	7
LWG defects	4190[37], 4206[38], 4215[39], 4356[40], 4368[41]	5
NB ballot	US 255-384[42], US 253-386[43], US 254-385[44], US 261-391[45]	4

4.2 Direction of Change

Every post-adoption item falls into one of three categories.

Origin	Items	Count
Sender Sub-Language	P2855R1, P2999R3, P3175R3, P3187R1, P3303R1, P3373R2, P3557R3, P3570R2, P3682R0, P3718R0, P3826R3, P3941R1, LWG 4190, 4206, 4215, 4368	16
Sender Integration	P3927R0, P3950R0, D3980R0, LWG 4356, US 255-384, US 253-386, US 254-385, US 261-391	8
Coroutine-Intrinsic	-	0

Twenty-four post-adoption items modified the sender sub-language or its integration. Zero modified coroutines.

5. Published Findings

Dietmar Kühl catalogued sixteen open concerns with `task` in P3796R1[7] ("Coroutine Task Issues," 2025). On symmetric transfer:

> *"The specification doesn't mention any use of symmetric transfer."*

Kühl reworked the allocator model in D3980R0[10] (2026-01-25) - six months after P3552R3[6]'s adoption at Sofia.

Jonathan Müller identified a stack overflow vulnerability in P3801R0[8] ("Concerns about the design of std::execution::task," 2025):

> *"Having iterative code that is actually recursive is a potential security vulnerability."*

Müller confirmed the cause:

> *"The reason `co_yield` is used, is that a coroutine promise can only specify `return_void` or `return_value`, but not both. If we want to allow `co_return;`, we cannot have `co_return with_error(error_code);`. This is unfortunate, but could be fixed by changing the language to drop that restriction."*

Robert Leahy proposed a core language change in P3950R0[9] (2025):

> *"Disallowing it either disadvantages coroutines vis-a-vis `std::execution` or necessitates library workarounds."*

Chris Kohlhoff identified the partial success tension in P2430R0[5] ("Partial success scenarios with P2300," 2021):

> *"Due to the limitations of the `set_error` channel (which has a single 'error' argument) and `set_done` channel (which takes no arguments), partial results must be communicated down the `set_value` channel."*

6. Structural Observations

These are not design defects. They are tradeoffs the sender model makes for compile-time work graphs. The question is whether those tradeoffs should also bind I/O.

Boundary	Sender Model Requires	Coroutine/I/O Cost
Error channels	Compile-time channel routing	`(error_code, size_t)` cannot route through three channels
Error returns	Separate `set_error` channel	`co_yield with_error(ec);` reverses established `co_yield` semantics
Frame allocator	Deferred execution via `connect` / `start`	`promise_type::operator new` fires before sender machinery
Symmetric transfer	Struct composition with void completions	`coroutine_handle<>` -returning `await_suspend` unreachable

Both P2300R10[1] bridges - `sender-awaitable` and `connect-awaitable` - use `void await_suspend`.

Senders get the allocator they do not need. Coroutines need the frame allocator they do not get.

6.1 The Error Channel Timeline

Date	Event
February 2020 (Prague)	Partial success raised during P1678R2[32] review. LEWG polls SF:7/F:14/N:9/A:3/SA:0. No resolution.
February 2021 (SG4 telecon)	Participant states sender/receivers have a loss: no success/partial-success.
July-October 2021 (LEWG)	Debated across five telecons. LEWG outcome document: ***"Better explain how partial success works with senders/receivers."***
August 2021	Kohlhoff publishes P2430R0[5].
2022-2024	Six revisions through P2300R10[1]. Three-channel model unchanged.
2023	Kühl's P2762R2[11] preserves Asio's `(error_code, size_t)` convention.
November 2023 (Kona)	SG4 polls that networking must use the sender model (SF:5/F:5/N:1/A:0/SA:1).

Date	Event
November 2024 (Wroclaw)	Channel question resurfaces during P0260 (Concurrent Queues). Debated across two face-to-face meetings.
February 2025 (Hagenberg)	Concurrent queue channel question remained open. Poll to reopen withdrawn.
2025-2026	P3570R2[19] documents `optional<T>` / channel model mismatch.

7. Established Practice

Library	Symmetric Transfer	Error Delivery	Async Model
cppcoro[53] (Baker)	`coroutine_handle<>`	`co_return` / exceptions	Coroutine-native
folly::coro[54] (Meta)	`coroutine_handle<>`	Exceptions	Coroutine-native
Boost.Cobalt[55] (Morgenstern)	`coroutine_handle<>`	`co_return` / exceptions	Coroutine-native
libcoro[56] (Baldwin)	`coroutine_handle<>`	`co_return` / exceptions	Coroutine-native
Boost.Asio (Kohlhoff)	N/A	Error codes via `as_tuple`	Completion-token
asyncpp[57] (Kardos)	Event-based	`co_return` / exceptions	Coroutine-native

SG14 (February 2026 mailing): ***"SG14 advise that Networking (SG4) should not be built on top of P2300. The allocation patterns required by P2300 are incompatible with low-latency networking requirements."***

Six independent libraries converged on `coroutine_handle<>`-returning `await_suspend`. The standard uses `void`.

8. Ecosystem Adoption

The sender/receiver model has been public since P2300R0[63] (2021). stdexec[49] has existed as a reference implementation throughout. These results may not be exhaustive; additions are welcome.

Domain	Project	Status
TCP networking	senders-io[50]	19 stars. "Still very experimental." Requires non-main stdexec branch.
HTTP	fuchsia[51]	2 stars, 0 forks. Last commit August 2023.
TLS	(none found)	
File I/O	senders-io[50]	Experimental. Documentation sections empty.
Database clients	(none found)	
DNS resolution	(none found)	
Signal handling	(none found)	
Bare metal / embedded	Intel cpp-baremetal-senders-and-receivers[52]	288 stars. P2300 subset. "Coroutines are not considered at all."
GPU / heterogeneous	stdexec[49]	1,300+ stars. Reference implementation. Active.
HFT / low-latency	(reported, proprietary)	Cannot be independently verified.

Rubén Pérez Hidalgo, author of Boost.MySQL, offered this assessment:

> *"This whole S/R thing is as if someone had seen Asio and said 'too simple'."*

The reference implementation has 1,300 stars. The I/O ecosystem built on it has 21.

9. Open Question

The domains where sender-based I/O could be publicly verified show no production implementations. The domains where deployment is reported are proprietary.

Acknowledgements

The author thanks Dietmar Kühl, Jonathan Müller, Chris Kohlhoff, Eric Niebler, and Lewis Baker for their published observations cited in this paper.

References

WG21 Papers

1. P2300R10 - ''std::execution'' (Michał Dominiak, Georgy Evtushenko, Lewis Baker, Lucian Radu Teodorescu, Lee Howes, Kirk Shoop, Michael Garland, Eric Niebler, Bryce Adelstein Lelbach, 2024). https://wg21.link/p2300r10
2. P0443R14 - ''A Unified Executors Proposal for C++'' (Jared Hoberock, et al., 2020). https://wg21.link/p0443r14
3. P0912R5 - ''Merge Coroutines TS into C++20'' (Gor Nishanov, 2018). https://wg21.link/p0912r5
4. P0913R1 - ''Add symmetric coroutine control transfer'' (Gor Nishanov, 2018). https://wg21.link/p0913r1
5. P2430R0 - ''Partial success scenarios with P2300'' (Chris Kohlhoff, 2021). https://wg21.link/p2430r0
6. P3552R3 - ''Add a Coroutine Task Type'' (Dietmar Kühl, Maikel Nadolski, 2025). https://wg21.link/p3552r3
7. P3796R1 - ''Coroutine Task Issues'' (Dietmar Kühl, 2025). https://wg21.link/p3796r1
8. P3801R0 - ''Concerns about the design of std::execution::task'' (Jonathan Müller, 2025). https://wg21.link/p3801r0
9. P3950R0 - ''return_value & return_void Are Not Mutually Exclusive'' (Robert Leahy, 2025). https://wg21.link/p3950r0
10. D3980R0 - ''Task's Allocator Use'' (Dietmar Kühl, 2026). https://isocpp.org/files/papers/D3980R0.html
11. P2762R2 - ''Sender/Receiver Interface For Networking'' (Dietmar Kühl, 2023). https://wg21.link/p2762r2
12. P3682R0 - ''Remove std::execution::split'' (Robert Leahy, 2025). https://wg21.link/p3682r0
13. P3481R5 - ''std::execution Concurrent Bulk'' (Eric Niebler, 2025). https://wg21.link/p3481r5
14. P3887R1 - ''when_all fix'' (in progress). https://wg21.link/p3887r1

15. P3396R1 - ‘‘std::execution wording fixes’’ (Eric Niebler, 2025). https://wg21.link/p3396r1
16. P3557R3 - ‘‘High-Quality Sender Diagnostics with Constexpr Exceptions’’ (Eric Niebler, 2025). https://wg21.link/p3557r3
17. P3433R1 - ‘‘Allocator support for std::execution’’ (2025). https://wg21.link/p3433r1
18. P3284R4 - ‘‘write_env and unstoppable’’ (Eric Niebler, 2025). https://wg21.link/p3284r4
19. P3570R2 - ‘‘Optional variants in sender/receiver’’ (Fabio Fracassi, 2025). https://wg21.link/p3570r2
20. P2079R10 - ‘‘System execution context’’ (2025). https://wg21.link/p2079r10
21. P3149R11 - ‘‘async_scope’’ (Ian Petersen, Jessica Wong, Kirk Shoop, et al., 2025). https://wg21.link/p3149r11
22. P3927R0 - ‘‘task_scheduler Support for Parallel Bulk Execution’’ (Lee Howes, 2026). https://wg21.link/p3927r0
23. P2855R1 - ‘‘Member customization points for Senders and Receivers’’ (Ville Voutilainen, 2024). https://wg21.link/p2855r1
24. P2999R3 - ‘‘Sender Algorithm Customization’’ (Eric Niebler, 2024). https://wg21.link/p2999r3
25. P3175R3 - ‘‘Reconsidering the std::execution::on algorithm’’ (Eric Niebler, 2024). https://wg21.link/p3175r3
26. P3187R1 - ‘‘Remove ensure_started and start_detached from P2300’’ (Kirk Shoop, Lewis Baker, 2024). https://wg21.link/p3187r1
27. P3303R1 - ‘‘Fixing Lazy Sender Algorithm Customization’’ (Eric Niebler, 2024). https://wg21.link/p3303r1
28. P3373R2 - ‘‘Of Operation States and Their Lifetimes’’ (Robert Leahy, 2025). https://wg21.link/p3373r2
29. P3718R0 - ‘‘Fixing Lazy Sender Algorithm Customization, Again’’ (Eric Niebler, 2025). https://wg21.link/p3718r0
30. P3826R3 - ‘‘Fix Sender Algorithm Customization’’ (Eric Niebler, 2026). https://wg21.link/p3826r3
31. P3941R1 - ‘‘Scheduler Affinity’’ (Dietmar Kühl, 2026). https://wg21.link/p3941r1
32. P1678R2 - ‘‘Callbacks and Composition’’ (Kirk Shoop, 2020). https://wg21.link/p1678r2
33. P4007R0 - ‘‘Senders and Coroutines’’ (Vinnie Falco, Mungo Gill, 2026). https://wg21.link/p4007r0
34. P4014R0 - ‘‘The Sender Sub-Language’’ (Vinnie Falco, Mungo Gill, 2026). https://wg21.link/p4014r0
35. P2583R1 - ‘‘Symmetric Transfer and Sender Composition’’ (Mungo Gill, Vinnie Falco, 2026). https://wg21.link/p2583r1
36. P4003R0 - ‘‘Coroutines for I/O’’ (Vinnie Falco, Steve Gerbino, Mungo Gill, 2026). https://wg21.link/p4003r0

LWG Issues

37. LWG 4190 - ‘‘completion-signatures-for specification is recursive.’’ https://cplusplus.github.io/LWG/issue4190
38. LWG 4206 - ‘‘connect_result_t should be constrained with sender_to.’’ https://cplusplus.github.io/LWG/issue4206

39. LWG 4215 - "run_loop::finish should be noexcept." https://cplusplus.github.io/LWG/issue4215
40. LWG 4356 - "connect() should use get_allocator(get_env(rcvr))." https://cplusplus.github.io/LWG/issue4356
41. LWG 4368 - "Potential dangling reference from transform_sender." https://cplusplus.github.io/LWG/issue4368

NB Ballot Comments

42. US 255-384. https://github.com/cplusplus/nbballot/issues/959
43. US 253-386. https://github.com/cplusplus/nbballot/issues/961
44. US 254-385. https://github.com/cplusplus/nbballot/issues/960
45. US 261-391. https://github.com/cplusplus/nbballot/issues/966

Blog Posts

46. Eric Niebler, "Structured Concurrency" (November 2020). https://ericniebler.com/2020/11/08/structured-concurrency/
47. Eric Niebler, "What Are Senders Good For, Anyway?" (February 2024). https://ericniebler.com/2024/02/04/what-are-senders-good-for-anyway/
48. Herb Sutter, "Living in the future: Using C++26 at work" (April 2025). https://herbsutter.com/2025/04/23/living-in-the-future-using-c26-at-work

Libraries

49. stdexec - NVIDIA reference implementation of std::execution (1,300+ stars). https://github.com/NVIDIA/stdexec
50. senders-io - Sender/receiver adaptation for async I/O (19 stars, "still very experimental"). https://github.com/maikel/senders-io
51. fuchsia - Experimental async networking on stdexec (2 stars, last commit August 2023). https://github.com/ColdOrange/fuchsia
52. Intel cpp-baremetal-senders-and-receivers - P2300 subset for embedded (288 stars, "coroutines are not considered at all"). https://github.com/intel/cpp-baremetal-senders-and-receivers
53. cppcoro - C++ coroutine abstractions (Lewis Baker). https://github.com/lewissbaker/cppcoro
54. folly::coro - Facebook coroutine library. https://github.com/facebook/folly/tree/main/folly/coro
55. Boost.Cobalt - Coroutine task types for Boost (Klemens Morgenstern). https://github.com/boostorg/cobalt
56. libcoro - C++20 coroutine library (Josh Baldwin). https://github.com/jbaldwin/libcoro
57. asyncpp - Async coroutine library (Péter Kardos). https://github.com/petiaccja/asyncpp

Platform Documentation

58. NVIDIA CUDA Programming Guide, Section 5.3 - "C++ Language Support." Coroutines not listed for device code compilation. https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/cpp-language-support.html

Other

59. C++ Working Draft - (Richard Smith, ed.). https://eel.is/c++draft/
60. LWG issues list - Library Working Group active issues. https://cplusplus.github.io/LWG/lwg-toc.html
61. WG21 paper mailings - ISO C++ committee papers. https://open-std.org/jtc1/sc22/wg21/docs/papers/
62. C++26 NB ballot comments - National body comments repository. https://github.com/cplusplus/nbballot
63. P2300R0 - "std::execution" (Eric Niebler, Kirk Shoop, Lewis Baker, Lee Howes, 2021). https://wg21.link/p2300r0